

TRIPLY — Stratégie de Tests

Application de planification de voyages personnalisée par IA

Version : 1.0

Date : Mai 2026

Type : Plan de test — Stratégie

Périmètre : MVP Triply — Toutes couches

0. Objectif du Plan de Test

Ce plan de test a pour objectif de garantir que Triply délivre des itinéraires fiables, personnalisés et adaptatifs. La nature IA-first de l'application implique des exigences de tests spécifiques allant au-delà d'une application classique.

Objectifs principaux :

- Expérience utilisateur fluide
- Recommandations pertinentes
- Gestion des imprévus
- Sécurité des données

Stack technique concernée :

Expo (React Native), Node.js backend, moteur IA (vectorisation + recommandations), APIs externes (météo, transport, hébergement, Mapbox), CI/CD GitHub Actions.

1. Méthodes de Test

1.1 Tests Unitaires

Les tests unitaires valident les composants isolés, notamment les algorithmes critiques.

Périmètre :

- Vectorisation du profil utilisateur (MBTI → vecteur)
- Moteur de recommandation
- Calcul d'itinéraire
- Cohérence météo / activité
- Scoring des hébergements
- Validation des données utilisateur

Exemple :

Un profil "Budget + Nature" doit générer des recommandations outdoor low-cost et exclure les activités premium.

Pourquoi c'est critique :

Toute erreur ici impacte directement la pertinence globale du produit.

1.2 Tests d'Intégration

Vérifient les interactions entre composants.

Flux testés :

- Frontend ↔ Backend
- Backend ↔ IA
- IA ↔ Base de données
- APIs externes (météo, transport, hébergement)
- Synchronisation temps réel

Exemple :

Une alerte météo doit :

1. notifier l'utilisateur
 2. proposer une alternative
 3. mettre à jour l'itinéraire
-

1.3 Tests Fonctionnels

Validation des fonctionnalités côté utilisateur.

Modules :

Profil & Personnalisation

- Questionnaire MBTI fonctionnel
- Recommandations cohérentes
- Mise à jour des préférences

Planification

- Génération d'itinéraire cohérente
- Respect contraintes (temps, budget, météo)
- Modification possible des étapes

Temps réel

- Modification d'activités
- Alertes météo et événements
- Gestion des réservations

Communauté

- Partage de voyage
 - Intégration des avis
-

1.4 Tests End-to-End (E2E)

Simulation complète du parcours utilisateur.

Parcours 1 : Nouveau utilisateur

1. Inscription
2. Questionnaire
3. Saisie des préférences
4. Génération d'itinéraire
5. Modification
6. Partage

Parcours 2 : Voyage en cours

1. Connexion
2. Carte interactive
3. Alerte météo
4. Modification
5. Consultation infos

Tests sur mobile (iOS/Android) et web.

1.5 Tests de Performance

Objectif : garantir rapidité et scalabilité.

Cibles :

- Itinéraire < 5s
- Recommandations < 2s
- Chargement app < 3s
- 500 utilisateurs simultanés
- API < 200ms

Scénarios :

- Charge normale
 - Pic saisonnier
 - Stress test
 - Endurance
-

1.6 Tests de Sécurité

Protection des données sensibles (RGPD).

Périmètre :

- Authentification sécurisée
- Autorisation API
- Chiffrement des données
- Protection contre injections
- Sécurisation géolocalisation
- Gestion des tokens

Point critique :

Les données de personnalité (MBTI) nécessitent une gestion RGPD stricte.

1.7 Tests de Régression

Évitent de casser l'existant.

Stratégie :

- Tests unitaires automatiques
 - Snapshots IA
 - Tests E2E avant merge
 - Comparaison de versions
-

1.8 Tests IA (Critiques)

Spécifiques au moteur intelligent.

Pertinence :

- Recommandations adaptées au profil
- Respect du budget
- Cohérence géographique

Cohérence :

- Distances réalistes
- Horaires valides
- Planning faisable

Stabilité :

- Résultats cohérents entre exécutions
 - Pas d'hallucinations
 - Cohérence saisonnière
-

2. Outils de Test

- Jest → tests unitaires
 - React Testing Library → tests frontend
 - Detox → E2E mobile
 - Playwright → E2E web
 - Postman / Newman → API
 - k6 → performance
 - OWASP ZAP → sécurité
 - GitHub Actions → CI/CD
 - Istanbul / nyc → couverture
-

3. Organisation des Tests

3.1 Pipeline CI/CD

- Unit tests → chaque commit
 - API tests → chaque commit
 - Régression → chaque PR
 - E2E → avant merge
 - Performance → avant déploiement
 - Sécurité → hebdomadaire
 - Tests IA → validation humaine
-

3.2 Priorisation

Critique :

- Moteur IA
- Pertinence recommandations
- Sécurité données

Haute :

- Modifications temps réel
- API météo
- Parcours complet

Moyenne :

- Performance
- Communauté
- Carte

3.3 Tests Manuels

- UX / ergonomie
 - Pertinence IA
 - Exploratoires
 - Compatibilité appareils
 - Accessibilité
-

Conclusion

La stratégie de test de Triply repose sur une approche complète adaptée à une application IA-first.

Points forts :

- Tests IA spécifiques
- Pipeline CI/CD complet
- Outils modernes
- Priorisation claire
- Conformité RGPD

Prochaines étapes :

- Implémenter les tests Jest
- Configurer Postman
- Mettre en place CI/CD
- Définir critères IA
- Tester sur devices réels